Reading a Papercraft Template: Self-Checking Vectorization of Low-Poly PDF Patterns for Cutting Machines

Paper ID xxx

Abstract

Low-poly papercraft models are shared as printable PDF templates: flat pieces with solid cut lines, dashed fold lines, glue tabs and small edge numbers that say which edges are glued together. A cutting machine needs vector paths instead, split into cut, score and draw operations. Generic raster-to-vector tools reproduce the picture but not its meaning: a dashed fold becomes many short cuts, and a misread edge number goes unnoticed. We present a pipeline that reads such a template as a fabrication plan. It separates the printed ink into pieces, folds and numbers, fits each piece as a polygon, reads the numbers with two independent readers, and then checks its own result with constraints the template carries for free: every number is printed exactly twice on edges of equal length, one of which is a tab; most pieces have a mirror twin; and the pieces must fold up into a closed surface. The constraints settle uncertain readings, repair traced geometry and decide which few questions are left for a person. On one 21-sheet mask template with 178 pieces and 1569 edge numbers, the unsupervised run pairs 67 % of the numbers with the right partner and traces outlines to a median of 0.6 mm; with the remaining questions answered, all 754 glued edges close in a 3D fold-up with a median gap of 0.1 mm. The output is one layered SVG file per model, editable in Inkscape and ready for a cutting machine.

CCS Concepts

• ***Computing methodologies*** → ***Image processing;*** • ***Applied computing*** → *Computer-aided manufacturing;*

1. Introduction

Papercraft masks and helmets made of low-poly cardstock are distributed by their designers, for sale or free for personal use, as PDF templates made for a home printer and scissors. Each page holds unfolded pieces drawn with three kinds of ink: solid cut lines, dashed (valley) and dash-dot (mountain) fold lines, and small edge numbers. Two edges that carry the same number are glued together, one of them through a tab. Many builders now own a cutting machine that can cut, score and draw on cardstock, but it needs vector paths assigned to those operations, and the template is a picture.

Raster-to-vector methods are judged by how close the result looks to the input [Sel03, HT06, NHS*13, BS19, EVA*20]. For a template this is the wrong measure. A fold traced as dashes becomes a row of cuts that perforate the piece; a missed corner moves a cut by a millimetre that no one sees but that keeps a seam from closing; and an edge number misread as a near-identical one sends the builder to the wrong piece. None of these errors changes the picture much.

Our observation is that a template is heavily redundant, and the redundancy can be checked without ground truth. Every edge number appears exactly twice, on two edges of equal length, one of which ends at a tab fold. Most pieces of a mask have a mirror twin with the same face. And the pieces, folded at their fold lines and glued at their numbers, must close into one surface. We build a vectorization pipeline (Figure 1) that uses these constraints in three ways: to choose between uncertain readings, to repair traced geometry, and to reduce human review to the cases the constraints leave open. Our contributions are:

- an ink-separation and polygon-fitting method for low-poly templates that produces cut polygons, single straight fold lines and OCR'd edge numbers instead of traced strokes;
- a two-reader number reader whose uncertain readings are resolved jointly by the pairing constraint;
- mirror-twin matching by sequence alignment of edge lengths, used to predict numbers, add missing folds and average corners;
- an evaluation on one 21-sheet template, with and without human answers.

The software is open source (link withheld for review). Our test case is a template found online, free for personal use and no longer maintained, whose small, low-quality edge numbers are hard to read by eye; the pipeline recovers them, and its 3D fold-up doubles as an interactive view of where each piece goes.

2. Related work

General vectorizers trace strokes or regions: Potrace fits curves to bitmap boundaries [Sel03], contour following [SA85] with Douglas–Peucker simplification [DP73] gives polygons, and line-drawing vectorization recovers stroke topology [HT06, NHS*13, BS19], also with learning on technical drawings [EVA*20].

© 2027 The Author(s).
Proceedings published by Eurographics - The European Association for Computer Graphics.
This is an open access article under the terms of the Creative Commons Attribution License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

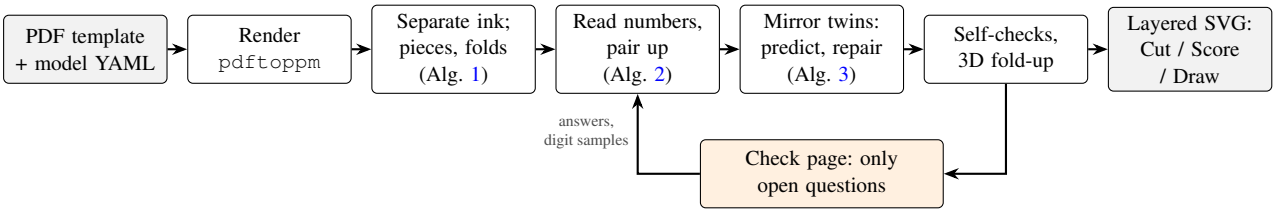


Figure 1: *Pipeline. Every stage after rendering works in millimetres on our own representation. The constraints of Section 3.3 are applied automatically; a person sees only what they cannot settle, and every answer is replayed on later builds and teaches the digit reader.*

Differentiable and generative vectorizers [LLGRK20, MZX*22, RAL*23] optimise visual similarity. None of them assigns fabrication meaning (cut versus fold, which edges glue) or checks it. Text in drawings is usually read by an OCR engine such as Tesseract [Smi07]; we add a page-trained digit matcher and a joint decision. Papercraft research mostly goes the other way, from a mesh to an unfolding [MS04, TWS*11]; we recover the unfolding's pieces and gluing from print. Our mirror matching is a 2D, combinatorial cousin of symmetry detection on shapes [MGP06], using sequence alignment [NW70] and a least-squares similarity fit [Ume91].

3. Method

The input is the PDF and a short YAML file per model that names the pages, the cardstock colour of each page and the template's fold styles. Pages are rendered at 180 dpi with `pdftoppm`; all geometry below is converted to millimetres with the page's own scale.

3.1. Pieces, cut polygons and folds

Printed ink is split by darkness and extent (Algorithm 1). Connected components longer than 4.3 mm are cut lines; dark short components are number glyphs; the remaining ink is fold dashes. A piece is what a flood fill of the paper from the page border cannot reach, which keeps numbers and dashes touching a cut from carving notches into it. Pieces may touch at a corner, so every enclosed open area seeds its own piece and grows through the cut ink.

Because the model is low-poly, a cut outline is a polygon with no curves. The contour is simplified, small bumps where a number touches a straight cut are removed, and each edge is refitted as a robust straight line through the printed cut ink near it, so that the edge follows the line, not the border of the filled region. Corners move to where neighbouring fitted lines meet, and vertices where the outline turns less than 4° are dropped.

Fold dashes inside a piece are found with a probabilistic Hough transform, merged when collinear (numbers printed on a fold interrupt its dashes), refitted to the dash pixels, and then stretched from the fold's midpoint to the first cut edge in each direction, snapping to a corner within 1.5 mm. A stretched fold must still be at least 60 % dashes, which rejects lines across a fan of folds. The fold's style is read from its dash runs: a dash-dot line has long runs where the dot merges with a dash. Each glyph group becomes a number mark attached to the nearest cut edge or fold it labels; a number on a fold marks a tab (Figure 2).

Algorithm 1 Pieces, cut polygons and folds of one page

Input: page raster I, scale s (mm/px)
Output: pieces P: cut polygon, folds, number marks

$C, G, D \leftarrow$ SplitInk(I) ▷ cut, glyph, dash masks
$O \leftarrow$ FloodFill($\neg$close(C), border) ▷ paper outside
for each enclosed region R of $\neg O$ **do**
  $Q \leftarrow$ ApproxPoly(contour(R), 1.5 px)
  $Q \leftarrow$ Straighten(Q) ▷ drop bumps from numbers
  repeat
    fit each edge of Q to C ink within a band (Huber)
    move each corner to its two fitted lines' intersection
    replace runs of ≤ 3 vertices whose edges leave C
  until no change
  drop vertices turning $< 4^\circ$; merge edges < 0.8 mm
  $F \leftarrow$ HoughSegments($D \cap R$); merge collinear
  for each $f \in F$ **do**
    extend f to the cut on both sides; snap ends to corners
    if dash coverage(f) < 0.6 **then** discard f
    style(f) $\leftarrow$ dashed or dash-dot from its run lengths
  $P \leftarrow P \cup \{(Q, F)\}$
group G into number marks; attach each to the nearest edge or fold

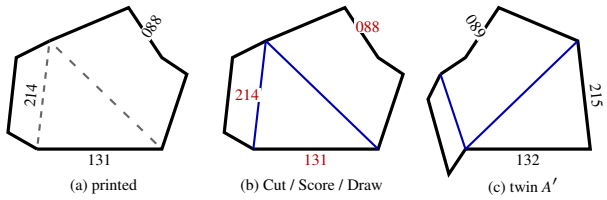


Figure 2: *A schematic piece (our drawing, not from the template). (a) Printed ink: solid cut, dashed folds, edge numbers; the number on a fold marks a tab. (b) Vector result: one cut polygon, one straight score line per fold, numbers as text. (c) Its mirror twin: the numbered lines have the same lengths in reverse order, while the tab may sit on another edge.*

3.2. Reading the edge numbers

Numbers are read, never traced. Each mark is rotated upright along its edge and read by two readers (Algorithm 2). Tesseract [Smi07] reads it as one line of digits in both orientations. A digit matcher is trained on the page itself: marks Tesseract reads with confidence $\geq$ 90 are split into digit cells, and every digit a person has corrected is added. A reading is accepted when both readers agree or Tesseract alone has confidence $\geq$ 93.

© 2027 The Author(s).
Proceedings published by Eurographics - The European Association for Computer Graphics.

Algorithm 2 Two readers and joint pair-up

Input: number marks M, hand answers H
Output: text and trust for every $m \in M$

for $m \in M$ **do** $(t_m, c_m) \leftarrow$ best Tesseract read over $0°, 180°$
train matcher on digits of $\{m : c_m \geq 90\} \cup H$
for $m \in M$ **do**
  $K_m \leftarrow$ matcher candidates (text, score), best first
  if $m \in H$ **then** accept H_m
  else if $t_m = K_m[1]$ **or** $c_m \geq 93$ **then** accept t_m
$n(x) \leftarrow$ number of accepted marks reading x
for undecided m by decreasing best score **do**
  $A \leftarrow \{k \in K_m : \text{score}(k) \geq \text{score}(K_m[1]) - 0.05\}$
  $x \leftarrow$ first $k \in A$ with $n(k) = 1$, else with $n(k) = 0$, else $K_m[1]$
  $\text{trust}(m) \leftarrow [n(x) = 1 \wedge x \in \{K_m[1], t_m\}]$; $n(x)$ += 1
flag every m whose text x has $n(x) \neq 2$

The remaining marks are decided together. Since every number must appear exactly twice, an uncertain mark, taken in order of the matcher's confidence, prefers a reading close to its best score that completes a pair, then one that starts a pair. It is trusted only when the completing reading was ranked first by one of the two readers.

3.3. Constraints that check the trace

3.3.0.1. Pairing. Every number now gets one status from its two copies: *ok* (one plain edge and one tab, lengths within 1 mm), *unpaired*, *length mismatch*, *no tab*, *both tabs* or *extra copy*. An unpaired number is matched to an unpaired line of the same length on the other side only when the reader offered that number as an alternative or the two differ in one digit, and no other candidate exists.

3.3.0.2. Mirror twins. A mask is mirror-symmetric, so most pieces have a twin cut from the same cardstock (Algorithm 3). Walking round one piece, the lengths of its numbered lines read the same as its twin's walked the other way, with gaps where a number found no line or a tab sits elsewhere. We align the two cyclic length sequences with a Needleman–Wunsch style recurrence [NW70] over all cyclic shifts, and accept a twin when at least 70 % of the lines agree within 1 mm. Twins are taken greedily by agreement. Matched line midpoints give a similarity transform with reflection [Ume91] that lays one twin on the other. Twins then give three checks and repairs: a line and its mirror line should be equally long; a fold of one twin with no fold at the mirrored place on the other is a missed fold, added in the style printed under it; and glue pairs mirror, so if line a of piece A glues to piece C, the mirror of a on A's twin glues to C's twin, which predicts its number. Mirror corners of the face that lie 0.2–1.5 mm apart both move to their mirrored mean; tab corners find no partner and stay, since a twin's tabs may sit on other edges.

3.3.0.3. Self-checks and fold-up. Further checks need no ground truth: a fold that meets the cut between two corners (a missed corner), cut ink that runs past a traced corner (a clipped corner), a piece traced twice, and a page whose glued pairs are all slightly too long or short (a scale error). The strongest check folds every piece at its fold lines into a 3D mesh, glues each numbered pair and measures the gap left; a misread number or a mistraced line shows up as a glue that does not close.

Algorithm 3 Mirror twins: matching, prediction, repair

Input: pieces with cut polygons, folds, numbered lines

for pairs (A, B), same cardstock, perimeters within 25 % **do**
  $\ell_A \leftarrow$ lengths of numbered lines of A in walking order
  $\ell_B \leftarrow$ same for B, *reversed*
  $(\sigma, \pi) \leftarrow \max_k \text{ALIGN}(\ell_A, \text{roll}(\ell_B, k))$
    ▷ gain 2 if $|\Delta| \leq 1$ mm, 1 if ≤ 3 mm
  keep (A, B, π) if share of agreeing lines ≥ 0.7
choose twins greedily by share, each piece at most once
for each twin (A, B, π) **do**
  $T \leftarrow$ similarity with reflection fit on matched midpoints
  flag line pairs in π differing > 1 mm in length
  add folds of A missing at $T(\cdot)$ on B, and vice versa
  move mutual-nearest corners 0.2–1.5 mm apart to their mean
  for unpaired line a of A with mirror $b = \pi(a)$ on B **do**
    if b glues cleanly to c on C and C has twin C' **then**
      predict number of a := number on $\pi_{CC'}(c)$

3.3.0.4. Human answers. What the constraints cannot settle is shown on a local check page as cards with the scan around both copies of a number and the traced lines. Answers are saved as CSV files (number corrections, hand-drawn lines, piece edits) that every later build applies before the constraints run, and each corrected number teaches the digit matcher.

3.4. Output

The result is one Inkscape SVG per model, with one layer per cardstock colour and one group per piece. Each group holds a *Cut* path (the polygon, tabs included), a *Score* group with one straight line per fold, split by valley and mountain style as the YAML names them, and a *Draw* group with the edge numbers as text. Ink outside every piece stays on a hidden *Review* layer, so nothing disappears silently. An export step converts to inches for the cutting machine's software and replaces the template's three-digit numbers by short labels.

4. Results

We ran the pipeline on one low-poly mask template, free for personal use [Pap17]: 24 US Letter pages, 21 of them with pieces on six cardstock colours. Table 1 compares the run with no human input to the build after all answers. Shapes are good without help: the unsupervised cut outlines lie a median 0.60 mm from the checked ones, 90 % within 1.4 mm, and only 3 of 178 pieces are more than 3 mm off. Mirror averaging alone made 262 corner moves and added 10 folds. The numbers are the bottleneck: the unsupervised run pairs 1059 of 1575 numbers with the right partner, 10 with a wrong one, and leaves 496 open, most of them tab-versus-edge confusions and misreads.

With all answers, 1548 of 1569 numbers pair cleanly and every one of the 754 glued edges closes in the fold-up, with a median gap of 0.1 mm (Figure 3). Of the 21 numbers left open, 6 are numbered edges around an eye opening that glue to nothing; the others

© 2027 The Author(s).
Proceedings published by Eurographics - The European Association for Computer Graphics.



Figure 3: *The reconstructed mask: our pieces, folded at their score lines and glued at their recovered numbers (front, three-quarter and side view, cardstock colours from the model YAML). All 754 glued edges close. The same fold-up drives an interactive viewer that links each 3D face to its flat piece. Mask design: Papercraft Cat (Remy Del Toro) [Pap17].*

Table 1: *One 21-sheet template, without and with human answers.*

	no answers	all answers
edge numbers found	1575	1569
trusted reads	970	1487
paired with right piece	1059	1548
paired with wrong piece	10	0
open	496	21
mirror twin pairs	86	89
3D fold-up: glues closing	527 / 631	754 / 754
3D fold-up: connected parts	3	1
human input (rows)	0	1079
number answers (changed)	–	670 (14)
hand-drawn lines	–	90
piece edits	–	319

are length mismatches and extra copies that do not keep the fold-up from closing. Of the 670 number answers only 14 changed the reading of the day, but they were not redundant: a reader trained without them reads 299 of the same marks differently. Replaying the answers in rounds of 100, least-trusted first, fixes about one pair per answer (1059, 1154, 1252, 1326, 1400 pairs after 0–400 answers), so roughly 550 answers would rebuild the checked result from scratch, half the 1079 rows the first build needed. An ablation of the automatic repairs is a warning: a third of the fixes applied without asking in the unsupervised run were wrong (38 of 112), so an automatic fix should need a second signal, a twin or the fold-up, and otherwise be shown as a suggestion.

5. Limitations and future work

The evaluation covers one template from one designer; other line styles need other thresholds in the YAML, and a multi-template benchmark with synthetic ground truth is in preparation. Reading is the weak stage. The 1548 confirmed numbers are a labelled set for the digit matcher, and a matcher trained on them should shorten the open list for the next template by the same designer; measuring this needs held-out pages. The constraints also suggest asking about pairs rather than single numbers: the fold-up proposes likely pairs by edge length and position, and confirming one answers two numbers.

The vector model makes changes possible that a printed template does not allow: wider glue tabs for the builder's adhesive, a different paper size, recolouring, re-segmenting the folded mesh into fewer pieces, and nesting the pieces on the fewest sheets. These are the subject of an extended paper; here we have shown that the template itself can be read reliably, because it checks itself.

References

[BS19] BESSMELTSEV M., SOLOMON J.: Vectorization of line drawings via polyvector fields. *ACM Transactions on Graphics 38*, 1 (2019), 9:1–9:12. 1

[DP73] DOUGLAS D. H., PEUCKER T. K.: Algorithms for the reduction of the number of points required to represent a digitized line or its caricature. *Cartographica 10*, 2 (1973), 112–122. 1

[EVA*20] EGIAZARIAN V., VOYNOV O., ARTEMOV A., VOLKHONSKIY D., SAFIN A., TAKTASHEVA M., ZORIN D., BURNAEV E.: Deep vectorization of technical drawings. In *Computer Vision – ECCV 2020* (2020), vol. 12358 of *Lecture Notes in Computer Science*, pp. 582–598. 1

[HT06] HILAIRE X., TOMBRE K.: Robust and accurate vectorization of line drawings. *IEEE Transactions on Pattern Analysis and Machine Intelligence 28*, 6 (2006), 890–904. 1

[LLGRK20] LI T.-M., LUKÁČ M., GHARBI M., RAGAN-KELLEY J.: Differentiable vector graphics rasterization for editing and learning. *ACM Transactions on Graphics 39*, 6 (2020), 193:1–193:15. 2

[MGP06] MITRA N. J., GUIBAS L. J., PAULY M.: Partial and approximate symmetry detection for 3D geometry. *ACM Transactions on Graphics 25*, 3 (2006), 560–568. 2

[MS04] MITANI J., SUZUKI H.: Making papercraft toys from meshes using strip-based approximate unfolding. *ACM Transactions on Graphics 23*, 3 (2004), 259–263. 2

[MZX*22] MA X., ZHOU Y., XU X., SUN B., FILEV V., ORLOV N., FU Y., SHI H.: Towards layer-wise image vectorization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2022), pp. 16314–16323. 2

[NHS*13] NORIS G., HORNUNG A., SUMNER R. W., SIMMONS M., GROSS M.: Topology-driven vectorization of clean line drawings. *ACM Transactions on Graphics 32*, 1 (2013), 4:1–4:11. 1

[NW70] NEEDLEMAN S. B., WUNSCH C. D.: A general method applicable to the search for similarities in the amino acid sequence of two proteins. *Journal of Molecular Biology 48*, 3 (1970), 443–453. 2, 3

[Pap17] PAPERCRAFT CAT (REMY DEL TORO): Bowser low-poly mask, papercraft template, 2017. PDF template, free for personal use. 3, 4

[RAL*23] RODRIGUEZ J. A., AGARWAL S., LARADJI I. H., RODRIGUEZ P., VAZQUEZ D., PAL C., PEDERSOLI M.: StarVector: Generating scalable vector graphics code from images, 2023. arXiv:2312.11556. 2

[SA85] SUZUKI S., ABE K.: Topological structural analysis of digitized binary images by border following. *Computer Vision, Graphics, and Image Processing 30*, 1 (1985), 32–46. 1

[Sel03] SELINGER P.: Potrace: a polygon-based tracing algorithm, 2003. Technical report, https://potrace.sourceforge.net/potrace.pdf. 1

[Smi07] SMITH R.: An overview of the Tesseract OCR engine. In *Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR)* (2007), pp. 629–633. 2

[TWS*11] TAKAHASHI S., WU H.-Y., SAW S. H., LIN C.-C., YEN H.-C.: Optimized topological surgery for unfolding 3D meshes. *Computer Graphics Forum 30*, 7 (2011), 2077–2086. 2

[Ume91] UMEYAMA S.: Least-squares estimation of transformation parameters between two point patterns. *IEEE Transactions on Pattern Analysis and Machine Intelligence 13*, 4 (1991), 376–380. 2, 3

© 2027 The Author(s).
Proceedings published by Eurographics - The European Association for Computer Graphics.